# TechPulse: Exploring Technology Discussions on Hacker News Through a Combined API-Based Information Service

Ioana Salaru

Faculty of Informatics,
Technical University of Vienna

**Abstract.** Public social-media APIs provide opportunities to study how people discuss technologies, products, and technical problems. This paper presents TechPulse, a lightweight information service implemented in Python and Streamlit for exploring public Hacker News discussions. The prototype combines the Algolia Hacker News Search API, which supports full-text search and filtering, with the official Hacker News Firebase API, which provides individual items, user profiles, and comment-tree relationships. Search results can be inspected in context through a Discussion Explorer, where comments are summarized using a small lexicon-based sentiment heuristic and frequent-term counts. A Trend Explorer compares bounded monthly samples for selected queries. The paper documents the API-selection process, the transformation from API responses to visual information, and a scenario-based evaluation workflow. Particular attention is given to practical lessons: the value of combining complementary APIs and the risks of interpreting simple text analytics as objective measurements. TechPulse is presented as an exploratory prototype rather than a representative measurement of the Hacker News community.

**Keywords:** Social media analysis · Hacker News · public APIs · information service · sentiment heuristic · trend exploration

## 1 Introduction

Social media platforms are useful sources for studying how communities discuss technologies, products, and emerging issues. Yet the information available to an application is not determined only by what users post. It is also shaped by the platform's data model, the available API endpoints, indexing choices, query semantics, access restrictions, and the transformations performed by the application. A prototype built around public APIs is therefore both an information service and a practical investigation into the limits of platform-mediated data.

TechPulse was developed for a Social Media practical assignment. Its purpose is to let a user move from a broad keyword search to a particular discussion and then compare related topics over time. Hacker News was selected as the source because it contains technology-oriented stories and user comments, and

because its public APIs allow data retrieval without an authentication setup. Security-related terms provide a meaningful direction for exploring technical conversations, while the application itself remains general enough to investigate other technology topics.

The main contribution is not a novel sentiment model or a production-ready analytics system. Instead, the paper documents the practical process of selecting and combining APIs, mapping their fields into an interactive application, and interpreting the results cautiously. The central questions are: (1) What does each API make easy or difficult? (2) How can search results be connected to the full discussion that produced them? (3) What can lightweight text summaries and sampled trends tell a user, and where do they become misleading?

The remainder of the paper is organized as follows. Section 2 describes the API-selection process and the two data sources. Section 3 presents the application workflow and data transformations. Section 4 defines a scenario-based evaluation. Section 5 discusses the observations and limitations and Section 6 concludes.

## 2 API Selection and Data Sources

### 2.1 Why Hacker News

Hacker News offered a different direction that aligned with the author's interest in software engineering and cybersecurity. Its public data is text-rich: stories have titles and links, while comments contain discussion and reactions. This makes it possible to build a workflow that combines retrieval, context, and lightweight analysis. The choice was also pragmatic: both the search service and the official API can be queried anonymously for the public data used by TechPulse. Avoiding authentication reduced setup friction.

### 2.2 Complementary APIs rather than a single source

TechPulse uses two services with different responsibilities. The Algolia HN Search API (https://hn.algolia.com/api) supports full-text search, filtering by item type, date and numeric fields, and result pagination [1]. It is the natural entry point when the user has a phrase but does not know an item identifier. The official Hacker News API (https://github.com/HackerNews/API) exposes individual items and user profiles under the Firebase-backed v0 endpoint [2]. It is useful after an item has been identified, because the item record includes fields such as its identifier, author, timestamp, text, score, and child-comment identifiers.

This division of responsibilities is central to the architecture. Algolia is used for discovery; Firebase is used to inspect an item and follow the comment tree. A combined design gives the prototype a more natural user experience than trying to force either API to perform both tasks.

### 2.3 Practical selection criteria

The selection can be summarized in five criteria: (i) access without an authentication flow; (ii) availability of textual content; (iii) ability to search by user-supplied terms; (iv) a stable identifier that connects search results to discussion records; and (v) sufficient documentation to understand response fields. Hacker News met these requirements. The choice also introduced a clear limitation: Hacker News is a particular technology-focused community, not a representative sample of all social media users. Findings from TechPulse must therefore be described as observations about the accessible Hacker News corpus, not about public opinion in general.

## 3 System Design and Data Processing

### 3.1 Application structure

TechPulse is implemented in Python using Streamlit for the interactive interface. The code is organized around API access and analysis utilities, with the main application coordinating user input, retrieval, transformations, and visual presentation. The API helper module contains search and item/profile/thread retrieval functions; the analysis utility module contains text cleaning, sentiment classification, and keyword counting. This separation makes it easier to distinguish data acquisition from interpretation, even though the prototype is intentionally small.

### 3.2 Search and result normalization

The Search view collects a query and optional constraints, such as content type, date range, minimum points, minimum comment count, sorting preference, and result limit. These values are translated into parameters for the Algolia endpoint. The response is JSON containing a list of hits and metadata such as the reported number of matches. Each hit is then converted into fields suitable for display: title or text, author, creation time, item type, score, comment count, and an item identifier. The identifier is important because it is the bridge between discovery and deeper inspection.

A result's visible text depends on its type. A story normally has a title and may link to an external article; a comment has text and belongs to a parent story or comment. As a result, the interface cannot assume that every hit has the same fields. Missing titles, deleted content, and absent optional values must be handled without crashing the view. Dates also need conversion from the representation returned by the API into a readable timestamp. The displayed result is therefore not a raw API response: it is a normalized presentation built from a subset of the response fields.

### 3.3 From a search hit to a discussion

A key workflow is Search → Discussion Explorer. When the selected result is a story, its item identifier identifies the discussion root. When the result is a comment, the application should use the associated story identifier when available, because the user normally wants to inspect the wider discussion rather than only the comment itself. The item identifier should remain visible in the interface so that users can understand what is being explored and can cross-check the record on Hacker News. This connection demonstrates why identifiers are more than implementation details: they preserve context across separate API calls and views.

### 3.4 Retrieving the comment tree

The official API represents comments as child identifiers associated with an item. To display a thread, the application starts from the root item and follows the identifiers of its children. TechPulse uses bounded breadth-first traversal, limiting both the depth and the number of comments retrieved. The bound is a deliberate trade-off: unrestricted traversal could generate many requests for a large discussion, while a shallow traversal is faster but may omit later branches. The interface should consequently describe its output as a retrieved portion of the thread, not necessarily every comment ever associated with the story.

The official API documentation notes that child relationships are represented through item identifiers and that a client must fetch items to inspect them [2]. This design is flexible but creates request overhead: one thread may require multiple network calls. Some child records may be missing, deleted, marked as dead, or temporarily unavailable. The application must tolerate those cases and avoid interpreting an incomplete thread as a complete representation of the conversation.

### 3.5 Text cleaning and analysis

Retrieved text is prepared before analysis. HTML markup is removed or normalized so that comments can be displayed as text. Keyword counting identifies frequent terms after tokenization and stop-word filtering. The output is a frequency summary of the retrieved text, not a semantic model: a frequent word may be central to the topic, but it may also be generic, ambiguous, or repeated because of a few long comments.

Sentiment is estimated using a small, manually curated positive and negative word lexicon. Tokens matching the positive set contribute positive evidence; tokens matching the negative set contribute negative evidence. The label is Positive when positive matches dominate, Negative when negative matches dominate, and Neutral when the counts tie or no sentiment words are found. This transparent rule is inexpensive and easy to explain, but it is not a validated sentiment-analysis model. It does not reliably understand negation, sarcasm, irony, context, domain-specific meanings, or the difference between criticism of a technology and criticism of a particular implementation.

### 3.6 Trend Explorer

The Trend Explorer samples search results for selected terms over monthly intervals and visualizes the returned sample sizes alongside the API-reported number of matches. This view is intended to support exploratory comparison, not to claim a complete count of every relevant post. Its interpretation depends on how each query is phrased, the time interval, the index's coverage, and the number of records requested per interval. In particular, a graph of retrieved records must not be labelled as total platform activity unless the retrieval process genuinely establishes that total.

### 3.7 Caching and interactive behavior

API calls and thread traversal can be repeated when a user changes a widget or revisits a view. Caching repeated retrievals can reduce redundant requests and improve responsiveness; Streamlit documents caching as a way to reuse the results of data queries and other expensive computations [3]. However, cached data can become stale, while disabling caching can increase repeated network traffic. This is a practical example of a design decision that affects both user experience and the freshness of the displayed information.

## 4 Scenario-Based Evaluation

### 4.1 Evaluation goal

The evaluation is a functional walkthrough rather than a controlled performance benchmark. The purpose is to determine whether the application supports a coherent exploration path and to record where API behavior or the analysis method changes the interpretation. No claim is made that the prototype measures the overall sentiment of Hacker News or detects every discussion matching a topic.

### 4.2 Case Study

To demonstrate the application's workflow, a search for the keyword *php* was used as a starting point. The default search parameters were retained.

Figure 1 shows the Search interface and the available query parameters. The results in Figure 2 include three comments with relatively high scores, ranging from approximately 100 to 600 points, and between 240 and 570 comments associated with their respective discussions. These results illustrate how the search functionality can help identify active discussions for further inspection.

The first comment shown in Figure 2 was then selected using the *Explore this discussion* function. This demonstrates how a result can be connected to the Discussion Explorer through its identifier, allowing the user to move from search results to the context of the corresponding discussion.

Figure 3 presents the resulting analysis. Approximately half of the retrieved comments are classified as neutral, while 35% are classified as negative and 15%

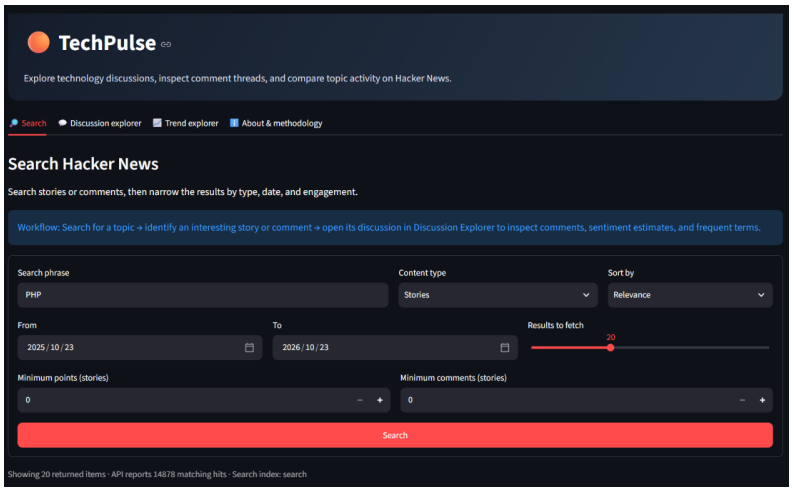



**Fig. 1.** Search interface with the query *php* and the available search parameters.

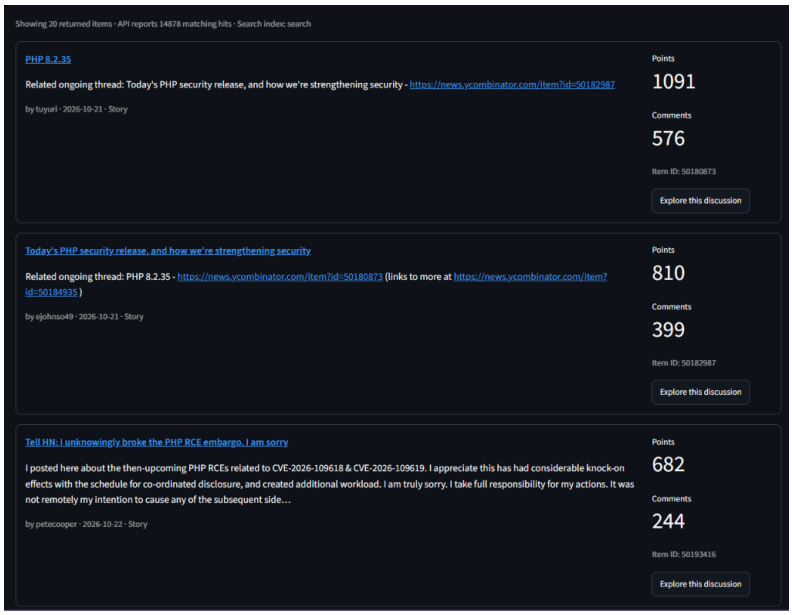



**Fig. 2.** Example search results showing comments with high scores and substantial discussion activity.

as positive. These categories are generated by the application's lexicon-based sentiment heuristic and should therefore be interpreted as approximate labels rather than definitive assessments of the authors' opinions.

The frequent-term chart also illustrates a limitation of simple keyword counting. Terms such as *now*, *oct*, *item*, and *yet*, as well as the term *matomo*, appear among the frequent words but do not necessarily summarize the main subject of the discussion. Word frequency alone does not establish topical relevance, and the usefulness of this chart depends on the retrieved text and the applied stop-word filtering.

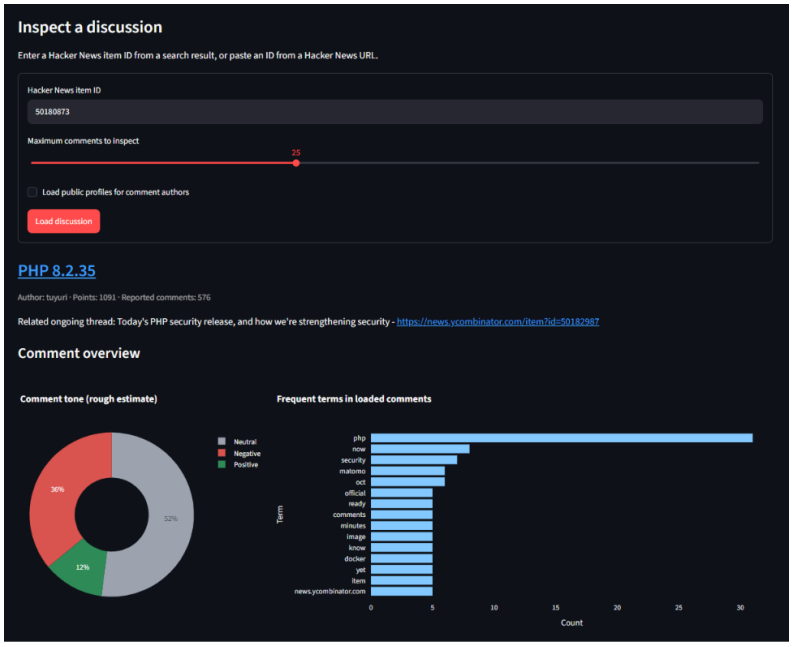



**Fig. 3.** Discussion Explorer showing the estimated sentiment distribution and frequent terms.

The retrieved comments can also be inspected directly, as shown in Figure 4. This view provides context for the automatically generated summaries and allows the user to compare the chart labels with the original text. In this way, the application supports exploratory inspection rather than relying exclusively on aggregated indicators.

Finally, the Trend Explorer was used to compare the queries *serverless*, *Kubernetes*, and *anomaly detection*. As illustrated in Figure 5, Kubernetes has the highest values in the displayed comparison and remains relatively stable across the observed period. Serverless shows higher values earlier in the year, followed

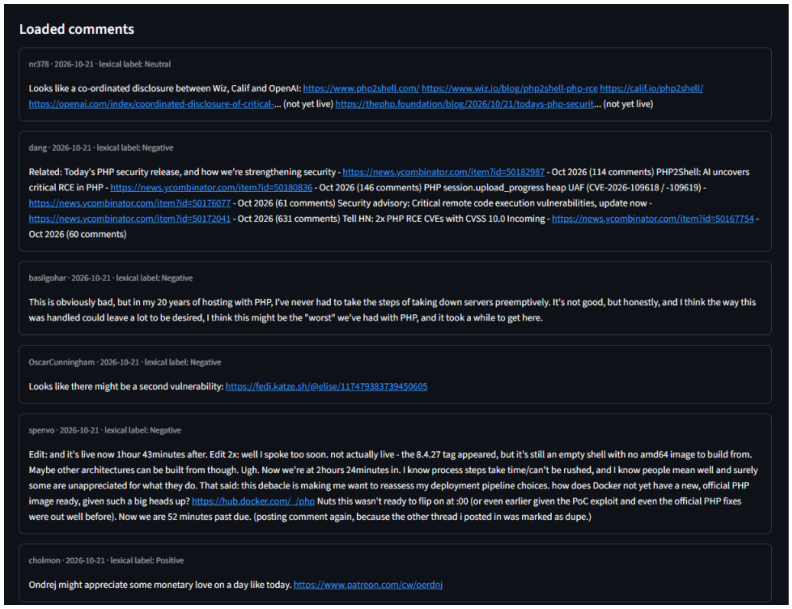



**Fig. 4.** Retrieved comments from the selected discussion.

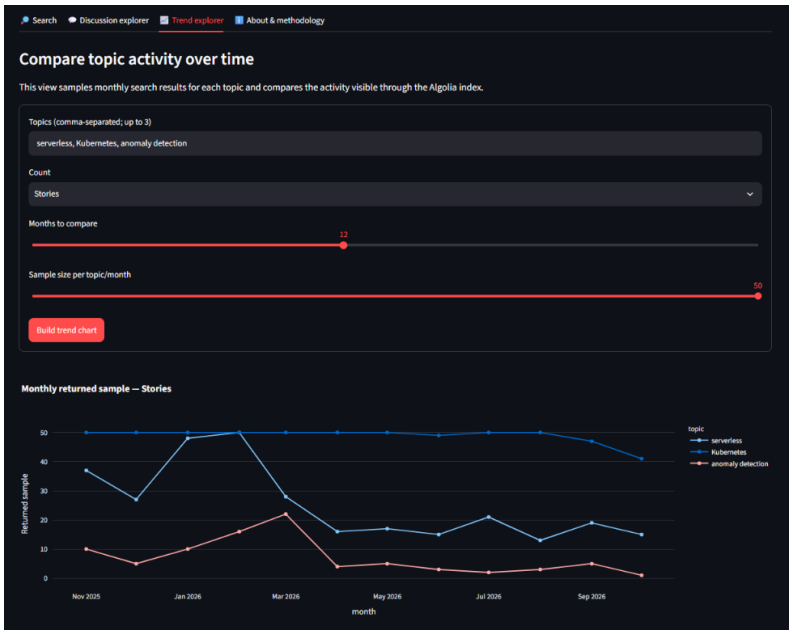



**Fig. 5.** Trend Explorer comparison of the queries *serverless*, *Kubernetes*, and *anomaly detection*.

by lower values in more recent periods. Anomaly detection has lower values overall but displays fluctuations over time.

## 5 Discussion: Lessons Learned and Limitations

*API composition.* The Algolia Search API and the official Hacker News API provide complementary capabilities. Search discovers relevant items from a query, while the official API retrieves item details and comment relationships. Combining them requires mapping identifiers, field names, and different data structures; API integration is therefore more than issuing HTTP requests.

*Search and indexing.* Keyword matches do not necessarily indicate topical relevance. Results depend on query wording, indexing, and the selected filters. Synonyms and related terms may be missed, while ambiguous terms can return unrelated content. Local filtering can narrow the returned results but cannot recover records that the API did not provide. The number of displayed hits should also be distinguished from the index's reported match count.

*Trend sampling.* Trend charts must be interpreted according to how data is retrieved. Bounded monthly samples depend on query wording, result limits, pagination, and sorting. A smaller sample does not necessarily indicate declining discussion volume. Even API-reported match counts reflect the search index and query semantics rather than a complete census of social activity. The plotted metric should therefore be stated explicitly.

*Comment structure.* Hacker News discussions form trees rather than flat lists. Replies may refer to parent comments, and traversal limits or unavailable records can exclude parts of a discussion. Flattening comments simplifies analysis but loses conversational context, which can affect the interpretation of sentiment and frequent terms.

*Sentiment and keyword analysis.* The lexicon-based sentiment method is fast, transparent, and requires no training data, but its small vocabulary cannot reliably capture context, negation, or sarcasm. Technical terms such as "failure", "attack", and "bug" do not necessarily indicate negative overall sentiment; for example, "the patch fixes the bug" describes a positive outcome. Sentiment labels should therefore be treated as rough indicators, not ground truth. Similarly, a frequent word is not necessarily a meaningful topic. If word colors represent the sentiment of comments containing each term, they describe those comments rather than the word's inherent meaning. Manual annotation could help evaluate the method, provided that the sample and annotation procedure are documented.

*Platform bias.* Hacker News is a technology-oriented community with its own language and participation patterns. Its users are not representative of the general public or all technology professionals. TechPulse supports exploration of Hacker News discussions, not general conclusions about public opinion.

*Improvements and trade-offs.* Potential improvements include a larger, domain-reviewed lexicon, validation against manually annotated comments, clearer indicators of retrieval coverage, query expansion, and export of the records used in each chart. More pagination or parallel requests could improve coverage but increase network traffic and complexity. A machine-learning sentiment model might capture more context, but would introduce additional dependencies, domain mismatch, and less transparent errors. Improvements should be chosen according to the prototype's goals rather than added solely for complexity.

## 6 Conclusion

TechPulse combines the Algolia HN Search API and the official Hacker News API to support exploratory analysis. It provides keyword search, comment-thread retrieval, frequent-term analysis, a simple sentiment heuristic, and sampled topic trends.

The main lesson is that public API access makes prototyping easier, but query design, indexing, sampling, missing data, and transformation choices shape the results. The visualizations help users navigate discussions and generate questions, but they do not provide a complete census of Hacker News or a reliable measure of community sentiment. Future work should validate summaries against manually inspected examples and report sampling and retrieval coverage more clearly. An information service should communicate both what its data supports and what it cannot establish.

## References

1. Algolia: Hacker News Search API. `https://hn.algolia.com/api` (accessed 23 October 2026).
2. Y Combinator: Hacker News API documentation. `https://github.com/HackerNews/API` (accessed 23 October 2026).
3. Streamlit: Caching overview. `https://docs.streamlit.io/develop/concepts/architecture/caching` (accessed 23 October 2026).
4. Streamlit: Session State. `https://docs.streamlit.io/develop/api-reference/caching-and-state/st.session_state` (accessed 23 October 2026).
5. Springer Nature: Lecture Notes in Computer Science author and proceedings templates. `https://www.springer.com/gp/computer-science/lncs` (accessed 23 October 2026).